DEFINICIÓN DE ARQUITECTURA Y ESPECIFICACIÓN TÉCNICA PARA EL MICROSERVICIO DE GESTIÓN DE INVENTARIO DE PRODUCTOS DE LIMPIEZA

1. Definición del Proceso de Negocio

El flujo operativo se divide en tres etapas críticas:

1. **Recepción y Validación**: Al llegar un producto, el operador registra los datos (nombre, SKU, proveedor). El sistema invoca al servicio externo de Validación de Representantes de Proveedores.
2. **Persistencia y Registro:** Una vez validado, el producto se guarda en la base de datos PostgreSQL con estado `PENDING_STOCK`.
3. **Sincronización de Ecosistema:** El microservicio emite un evento `products-events` hacia Apache Kafka. El mismo microservicio consumen este evento. El stock se actualiza dinámicamente consumiendo mensajes del tópico `products-events`.

2. Diseño de la Arquitectura (Arquitectura en Capas)

Complementada con patrones específicos de Quarkus y Hibernate Panache. Esta estructura es ideal para microservicios porque separa las responsabilidades, facilitando el mantenimiento y las pruebas unitarias.

A continuación se describe cómo se distribuye cada nivel en el proyecto:

1. Capa de Exposición o Presentación (ProductResource)

Es el punto de entrada de la aplicación. Su única responsabilidad es gestionar el protocolo HTTP.

- **Funciones**: Define los endpoints (@Path), recibe los datos del cliente, invoca a la capa de servicio y devuelve la respuesta con el código de estado adecuado.
- **Aislamiento**: Gracias al uso de DTOs, esta capa nunca expone la estructura interna de la base de datos al cliente.

2. Capa de Negocio o Servicio (ProductService / ProductServiceImpl)

Es el "cerebro" de la aplicación. Aquí es donde reside la lógica.

- **Funciones**: Orquestación de procesos, validaciones de negocio complejas y gestión de transacciones (@Transactional).
- **Transformación**: Utiliza el ProductMapper para convertir los datos que vienen de la persistencia (Entidades) en datos para la API (DTOs).

3. Capa de Persistencia o Datos (ProductEntity)

En tu caso, esta capa implementa el patrón Active Record a través de PanacheEntityBase.

- **Funciones**: A diferencia de la arquitectura tradicional con "Repositorios" separados, aquí la propia entidad contiene los métodos de acceso a datos (listAll, persist, findById).
- **Mapeo**: Define cómo se guardan los datos en las tablas de la base de datos mediante anotaciones JPA.

3. Especificaciones Técnicas y Stack

Componentes Core

- **Runtime:** Java 21. Se utilizarán `@RunOnVirtualThread` para optimizar el I/O bloqueante de Hibernate sobre los hilos portadores.
- **Framework:** Quarkus 3.x.
- **Persistencia:** Hibernate con Panache (Repository Pattern) + PostgreSQL.
- **Mapping:** MapStruct para conversión `Entity <-> DTO`.
- **Validación:** Hibernate Validator (JSR 380).

Ejemplo de Configuración (`application.yml`)

```yaml
quarkus:
 rest-client:
   escuelajs-api:
     url: https://api.escuelajs.co
     scope: jakarta.inject.Singleton
 http:
   port: 8082
 datasource:
   username: postgres
   password: 123456789
   jdbc:
     url:
jdbc:postgresql://localhost:5432/db_deepclean_product
 hibernate-orm:
   database:
     generation: update
   log:
     sql: true
     format-sql: true
     bind-parameters: true

 # DESACTIVAR DEV SERVICES COMPLETAMENTE
 kafka:
   devservices:
     enabled: false

# Configuración de Kafka Local
kafka:
```

```
 bootstrap:
   servers: localhost:9092

mp:
 messaging:
   outgoing:
     product-out:
       connector: smallrye-kafka
       topic: products-events
       value-serializer:
io.quarkus.kafka.client.serialization.ObjectMapperSerializer

   incoming:
     product-in:
       connector: smallrye-kafka
       topic: products-events
       value-deserializer:
com.deep.clean.app.kafka.ProductEventDeserializer
```

4. Implementación del Modelo y Persistencia

Utilizaremos el **Repository Pattern** para separar la lógica de persistencia de la entidad de dominio.

Entidad de Dominio

```
@Entity
@Table(name = "products")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ProductEntity extends PanacheEntityBase {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @NotBlank(message = "SKU no puede estar vacío")
   @Column(unique = true, nullable = false)
   private String sku;

   @NotNull(message = "El nombre no puede ser nulo")
   @Column(nullable = false)
```

```java
    private String name;

    @Column(length = 500)
    private String description;

    @Column(name = "category_id")
    private Integer categoryId;

    @PositiveOrZero(message = "El stock actual no puede ser
negativo")
    @Column(name = "current_stock")
    private Integer currentStock;

    @PositiveOrZero(message = "El umbral de stock mínimo no
puede ser negativo")
    @Column(name = "min_stock_threshold")
    private Integer minStockThreshold;

    @NotNull(message = "El estado no puede ser nulo")
    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private ProductStatus status;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        this.updatedAt = LocalDateTime.now();
    }
}
```

```java
@Entity
@Table(name = "product_logs")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
```

```java
@Builder
public class ProductLogEntity extends PanacheEntityBase {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long productId;
    private String sku;
    private String action;
    private String description;
    private LocalDateTime timestamp;
}
```

Repositorio (Panache)

```java
@ApplicationScoped
public class ProductRepository implements
PanacheRepositoryBase<ProductRepository, Long> {
}
```

5. Integración y Contratos API

Endpoint de Registro (REST)

Se implementa un `ExceptionMapper` global para estandarizar los errores (400 Bad Request, 404 Not Found, 500 Internal Error).

```java
@Path("/api/v1/products")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class ProductResource {

    @Inject
    ProductService productService;

    @Inject
    ProductProducer productProducer;

    @GET
    @Path("/all")
    public Response getAll() {
        return Response.status(Response.Status.OK)
```

```
                .entity(productService.findAll()).build();
    }

    @GET
    @Path("/find/{id}")
    public Response getById(@PathParam("id") Long id) {
        return Response.status(Response.Status.OK)
                .entity(productService.findById(id)).build();
    }

    @POST
    @Path("/create")
    public Response create(@Valid ProductDTO productDto) {
        // 1. Persistir (Si falla aquí, lanza
ConflictException y no llega a Kafka)
        ProductDTO created =
productService.create(productDto);

        // 2. Notificar (Solo si la transacción de DB terminó
exitosamente)
        productProducer.sendProductEvent(
                created.getId(),
                created.getSku(),
                created.getName(),
                created.getStatus() != null ?
created.getStatus().name() : "ACTIVE"
        );

        return
Response.status(Response.Status.CREATED).entity(created).buil
d();
    }

    @PUT
    @Path("/update/{id}")
    public Response update(@PathParam("id") Long id, @Valid
ProductDTO productDto) {
        ProductDTO updated = productService.update(id,
productDto);

        productProducer.sendProductEvent(
                updated.getId(),
                updated.getSku(),
                updated.getName(),
                updated.getStatus() != null ?
updated.getStatus().name() : "ACTIVE"
        );

        return Response.ok(updated).build();
    }
```

```
    @DELETE
    @Path("/delete/{id}")
    public Response delete(@PathParam("id") Long id) {
        productService.delete(id);
        return Response.noContent().build();
    }
}
```

```
@Path("/api/v1/supplier-representative")
public class SupplierRepresentativeResource {

    @Inject
    SupplierRepresentativeServiceImpl
supplierRepresentativeService;

    @GET
    @Path("/info-representatives")
    public Response getSupplierRepresentativeInfo() {
        return Response.ok(

supplierRepresentativeService.getInfo()).build();
    }

}
```

6. Flujo de Eventos (Kafka)

Para garantizar la **consistencia eventual**, el servicio emite un mensaje solo después de que la transacción de base de datos se haya completado satisfactoriamente.

Productor de Eventos

```
@ApplicationScoped
public class ProductProducer {

    private static final Logger LOG =
Logger.getLogger(ProductProducer.class);

    @Inject
    @Channel("product-out")
    Emitter<ProductEvent> eventEmitter;
```

```
   public void sendProductEvent(Long id, String sku, String
name, String status) {
       ProductEvent event = ProductEvent.builder()
               .id(id)
               .sku(sku)
               .name(name)
               .status(status)
               .timestamp(LocalDateTime.now().toString())
               .build();

       LOG.infof("Emitiendo evento Kafka para producto: %s",
sku);
       eventEmitter.send(event);
   }
}
```

Consumidor de Eventos

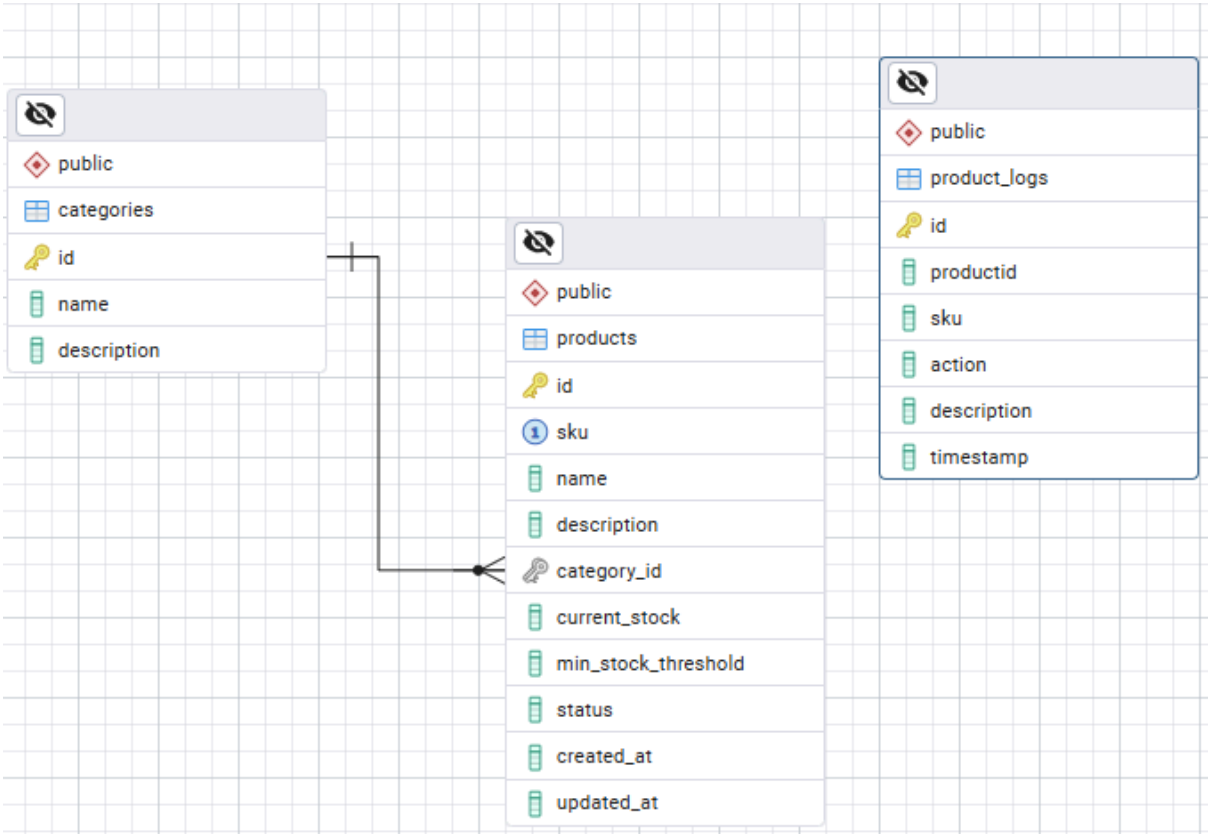
```
@ApplicationScoped
public class ProductConsumer {

   private static final Logger LOG =
Logger.getLogger(ProductConsumer.class);

   @Incoming("product-in")
   @Transactional
   public void consumeProductEvent(ProductEvent event) {
       LOG.infof("==> Kafka Consumer: Procesando evento para
el producto SKU: %s", event.getSku());

       // Guardamos en la bitácora de la base de datos
       ProductLogEntity logEntry = ProductLogEntity.builder()
               .productId(event.getId())
               .sku(event.getSku())
               .action("EVENT_RECEIVED")
               .description("Evento de producto procesado
desde Kafka: " + event.getName())
               .timestamp(LocalDateTime.now())
               .build();

       logEntry.persist();

       LOG.infof("==> Kafka Consumer: Log persistido con
éxito para SKU: %s", event.getSku());
   }
}
```

7. Validaciones y Seguridad

- **Integridad:** Bean Validation en DTOs para prevenir entradas malformadas.
- **Auditabilidad:** Uso de campos `@CreationTimestamp` y `@UpdateTimestamp` en las entidades.
- **Resiliencia:** Implementación de *Retries* en el `RestClient` para la validación de proveedores mediante `@Retry(maxRetries = 3)`.

> **Nota del Arquitecto:** La elección de **Quarkus + Java 21** nos permite reducir el consumo de memoria en un **40%** comparado con arquitecturas tradicionales de Spring Boot, ideal para despliegues en contenedores (K8s) con alta densidad de microservicios.

8. Diseño y creación de base de datos




```
-- Table: public.products

-- DROP TABLE IF EXISTS public.products;

CREATE TABLE IF NOT EXISTS public.products
(
```

```
    id bigint NOT NULL DEFAULT nextval('products_id_seq'::regclass),
    sku character varying(255) COLLATE pg_catalog."default" NOT NULL,
    name character varying(255) COLLATE pg_catalog."default" NOT NULL,
    description character varying(500) COLLATE pg_catalog."default",
    category_id integer,
    current_stock integer DEFAULT 0,
    min_stock_threshold integer DEFAULT 5,
    status character varying(255) COLLATE pg_catalog."default" NOT NULL,
    created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    updated_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT products_pkey PRIMARY KEY (id),
    CONSTRAINT products_sku_key UNIQUE (sku),
    CONSTRAINT products_category_id_fkey FOREIGN KEY (category_id)
        REFERENCES public.categories (id) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.products
    OWNER to postgres;
```

```
-- Table: public.categories

-- DROP TABLE IF EXISTS public.categories;

CREATE TABLE IF NOT EXISTS public.categories
(
    id integer NOT NULL DEFAULT nextval('categories_id_seq'::regclass),
    name character varying(100) COLLATE pg_catalog."default" NOT NULL,
    description text COLLATE pg_catalog."default",
    CONSTRAINT categories_pkey PRIMARY KEY (id)
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.categories
    OWNER to postgres;
```

```
-- Table: public.product_logs

-- DROP TABLE IF EXISTS public.product_logs;

CREATE TABLE IF NOT EXISTS public.product_logs
```

```
(
    id bigint NOT NULL DEFAULT nextval('product_logs_id_seq'::regclass),
    productid bigint,
    sku character varying(255) COLLATE pg_catalog."default",
    action character varying(255) COLLATE pg_catalog."default",
    description character varying(255) COLLATE pg_catalog."default",
    "timestamp" timestamp without time zone,
    CONSTRAINT product_logs_pkey PRIMARY KEY (id)
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.product_logs
    OWNER to postgres;
-- Index: idx_product_logs_sku

-- DROP INDEX IF EXISTS public.idx_product_logs_sku;

CREATE INDEX IF NOT EXISTS idx_product_logs_sku
    ON public.product_logs USING btree
    (sku COLLATE pg_catalog."default" ASC NULLS LAST)
    WITH (fillfactor=100, deduplicate_items=True)
    TABLESPACE pg_default;
```